

College Academic Resource Management System

Project Information & Technical Documentation — generated 16 August 2026

1. Project Overview

The College Academic Resource Management System (RGCET) is a secure, role-based web platform for managing academic resources, students, subjects and staff in a multi-department college. Each department operates in a fully isolated data domain: users belonging to one department can never view, download or modify records belonging to another department. The system provides resource upload and secure download, student record management, batch Excel import/export, department-level reports, an audit trail, and a JWT-protected REST API.

2. Key Features

Feature	Details
New User page	Creates teacher accounts with name, phone, email, department, password + confirm password
Login-page registration	Teacher/HOD accounts can be created from the login page — auto-login after signup (Super Admin never allowed)
Multi-subject teachers	One teacher can be assigned to many subjects at creation and in dashboards
Teacher dashboard	Lists every assigned subject with live per-subject resource counts
HOD full department view	HOD sees all staff, subjects, students-by-year and resources of their department
Department isolation	Cross-department access is blocked (403 web / 404 API)
Excel import/export	Batch student import with column mapping and filtered exports
Reports & audit trail	Department/college-wide reports and full activity logging

3. Technology Stack

Layer	Technology	Purpose
Backend	Python 3.13 / Django 5.2	Core framework, ORM, admin, auth
REST API	Django REST Framework 3.16	API endpoints for each module
Auth / JWT	SimpleJWT	Access + refresh tokens for the API
Database	MySQL 8.x	Production data store
Test DB	SQLite (in-memory)	Isolated test runs
Templates	Django templates + Bootstrap 5	Responsive UI, sidebar layout
Excel	pandas / openpyxl	Batch student import and export
PDF	reportlab	Documentation PDF generation
Files	Pillow	Image handling for profiles
Env	python-dotenv	Configuration via .env

4. Architecture & Applications

App	Responsibility
config	Settings split into base / development / production / testing
accounts	Users, StaffProfile, roles, login, New User page, staff management, auth mixins
departments	Department CRUD (Super Admin only)
subjects	Subjects, academic years, regulations, teacher assignments (multi-subject)
resources	Resource categories and department-scoped file uploads
students	Student records, filters, Excel export
excel_manager	Two-step Excel import with column mapping and validation
dashboard	Role-based dashboards (multi-subject teacher view, full HOD department view)
audit	Audit log of every sensitive action
core_api	JWT-protected REST ViewSets with department scoping

5. Roles & Access Control

Role	Scope
Super Admin	Full access across all departments; manages departments, staff and New User creation
HOD	Own department only — sees all staff, subjects, students and resources of the department
Teacher	Own department; can be assigned multiple subjects and sees them on the dashboard

6. Security Model

Isolation is enforced at every layer, not just hidden in the UI:

Mechanism	Description
DepartmentAccessMixin	Denies object access across departments (403 on web)
TeacherSubjectAccessMixin	Teachers blocked from subjects they are not assigned to
Role mixins	RoleRequired / HODOrAbove / SuperAdminRequired gate views
API scoping	REST ViewSets filter by department; cross-dept returns 404
Secure downloads	Files never served via /media/ — always through an authorized view
Audit trail	Uploads, downloads, imports and logins recorded with IP + user
Deactivation	Deactivated accounts cannot log in or access any page

7. Key URLs

URL	Purpose
/	Redirect to dashboard
/accounts/	Login, profile, staff management
/dashboard/	Role dashboards + reports
/departments/	Department CRUD (Super Admin)
/subjects/	Subject CRUD + teacher assignments
/resources/	Resource upload / browse / download
/students/	Student records + Excel export

URL	Purpose
/excel/	Batch Excel import
/audit/	Audit trail
/api/v1/	REST API (JWT)
/admin/	Django admin

8. Running the Project

Setup: create MySQL database *college_management*, copy *.env.example* to *.env* and configure credentials, then run *pip install -r requirements.txt*, *python manage.py migrate*, *python manage.py seed_demo_data* and *python manage.py runserver*.

Testing: *python manage.py test --settings=config.settings.testing* (36 tests, SQLite in-memory).

Docs: see README.md and USER_CREDENTIALS.pdf for the seeded login accounts.